Water Quality Prediction using Bayesian Belief Network

Assignment 2 — PS10 | Group 90

Course: Artificial and Computational Intelligence

Date: August 6, 2026

1. Problem Overview

Water potability is a binary classification problem: given nine physicochemical measurements, determine whether water is safe for human consumption (Potability = 1) or not (Potability = 0). The dataset (`water_potability.csv`) contains 3,276 samples with attributes: pH, Hardness, Solids, Chloramines, Sulfate, Conductivity, Organic Carbon, Trihalomethanes, and Turbidity.

2. Chosen Approach — Bayesian Belief Network (BBN)

2.1 Rationale

A Bayesian Belief Network (BBN) is a probabilistic graphical model that encodes conditional dependency relationships between variables via a Directed Acyclic Graph (DAG). It is well-suited for this problem because:

- It produces interpretable probability distributions over Potability rather than a hard label.
- It supports inference queries: predict Potability given a subset of observed variables.
- It naturally handles incomplete evidence (not all attributes need to be observed).

2.2 Network Structure

A **Naïve Bayes topology** is used — a simplification where all nine attribute nodes are direct parents of the single Potability node:

This structure is a well-studied approximation for high-dimensional data with unknown inter-attribute dependencies.

2.3 Discretization

BBNs require discrete state spaces. Continuous attributes are discretized into three equal-frequency bins using `pd.qcut`:

Category	Rule
`low`	value ≤ 33rd percentile
`medium`	33rd < value ≤ 67th percentile
`high`	value > 67th percentile

Rows containing NaN values (≈ 15% of the dataset) are dropped after discretization to ensure clean CPD learning.

2.4 Parameter Learning

Conditional Probability Distributions (CPDs) are learned from the discretized training data using **Bayesian Estimation** with symmetric Dirichlet priors (pseudo-count = 1 per state). This is equivalent to Laplace smoothing and prevents zero-probability issues in the learned CPDs.

A Maximum Likelihood Estimation fallback is included in case the Bayesian estimator cannot converge.

2.5 Inference

At query time, `pgmpy`'s **Variable Elimination** algorithm is used. It marginalises out all unobserved nodes to compute `P(Potability | evidence)` exactly.

3. Implementation Workflow

```
Load CSV  →  Drop header NaNs  →  Discretize (qcut)  →  Drop NaN rows
     →  Fit BayesianNetwork (pgmpy)  →  VariableElimination
     →  Q2: predict_potability()
     →  Q3: infer_probability()
     →  Q4: infer_conditional_probability()
     →  Write outputPS10.txt
```

Input reading: `inputPS10.txt` is parsed line-by-line; lines starting with `#` are skipped. Keys and float values are extracted to build the evidence dictionary for Q2.

Output writing: The output file is deleted at the start of each run (via `os.remove`) to prevent stale data accumulation, then rebuilt with clearly labelled sections for Q2, Q3, and Q4.

4. Problem statements & solutions

#	Query	Method
Q2	Predict Potability for given ph, Hardness, Solids, Chloramines, Sulfate, Conductivity, Turbidity	`predict_potability()` — categorises inputs, runs VE
Q3	Infer P(Potability) given all 8 non-ph attributes	`infer_probability()` — Potability excluded from evidence

#	Query	Method
Q4	P(Potability = 1 | ph=low, Hardness=high, Solids=high)	`infer_conditional_probability()` — direct categorical evidence

5. Alternate Modeling Approach — Random Forest Classifier

An alternate approach is a **Random Forest (RF)** ensemble classifier using `sklearn.ensemble.RandomForestClassifier`.

5.1 How it differs

Aspect	BBN (chosen)	Random Forest (alternate)
Data handling	Requires discretization; drops NaN rows	Handles continuous values natively; supports imputation
Output	P(Potability | evidence) — full distribution	P(Potability) via `predict_proba`
Interpretability	Explicit CPDs; causal semantics	Feature importances; no causal model
Partial evidence	Naturally handles missing attributes in query	All features required at inference
Training complexity	O(n × k) for CPD counting	O(n × d × T × log n) for T trees
Inference complexity	Exponential in treewidth (manageable for naïve structure)	O(T × d) per query — faster

5.2 Performance implications

- **Accuracy:** Random Forest typically achieves higher predictive accuracy (~75–80% on this dataset) versus a naïve Bayes BBN (~60–65%) because RF can model non-linear interactions between attributes without the conditional independence assumption.
- **Memory:** RF stores T full decision trees; BBN stores only CPD tables (far smaller).
- **Inference speed:** RF is faster at query time (milliseconds). BBN Variable Elimination on this structure is also fast (linear in the number of parents), but would become expensive for denser graph structures.
- **Interpretability:** BBN wins — CPDs provide human-readable probability tables and support "what-if" probabilistic queries that RF cannot answer natively (e.g., Q4).

Conclusion: RF is preferable for pure classification accuracy. BBN is preferable when probabilistic reasoning, partial evidence handling, and interpretability are priorities — which aligns with the assignment's query requirements.

6. Key Design Decisions

1. **Naïve Bayes structure over structure learning** — Structure learning algorithms (Hill-Climb, PC) require significantly more data to produce stable graphs; the naïve structure is a robust baseline given the dataset size.

2. **Equal-frequency binning (qcut)** — Ensures roughly equal support per category, avoiding empty CPD cells that would require heavy smoothing.
3. **Dirichlet prior (pseudo-count = 1)** — Prevents zero probabilities in CPDs without biasing estimates significantly given ~2,700 training samples post-NaN removal.

7. Libraries Used

Library	Version	Purpose
`pgmpy`	≥ 0.1.21	BBN construction, parameter learning, inference
`pandas`	≥ 1.3	Data loading, discretization
`numpy`	≥ 1.21	Numerical operations
